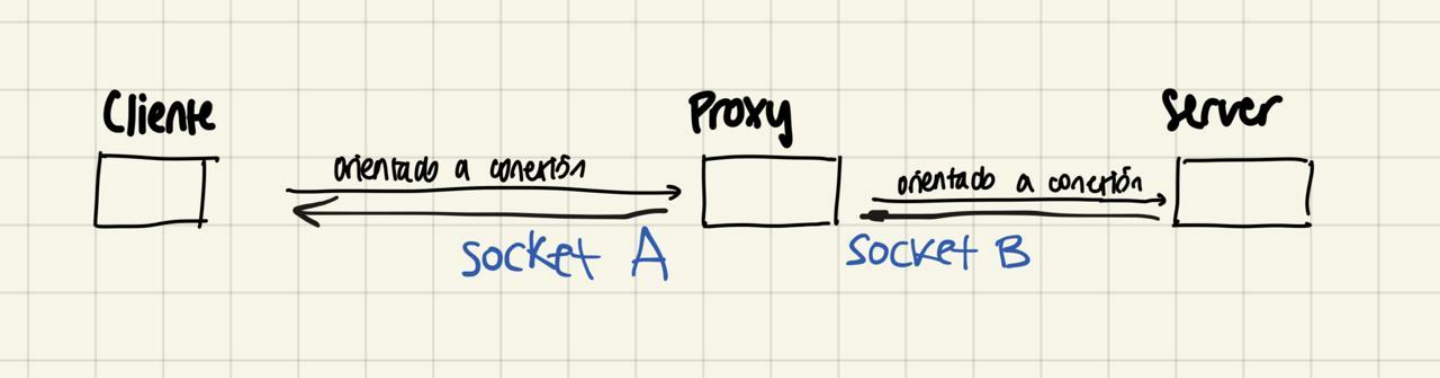
INFORME ACTIVIDAD HTTP

NICOLE CHARUN ALARCON

DIAGRAMA



El flujo funciona en este orden:

1. Cliente → Proxy (socket A): El cliente se conecta al proxy mediante un socket orientado a conexión (TCP). En el código se ve como "A". El cliente envía su request al proxy y espera respuesta. El proxy acá actúa como "Servidor" para el cliente.
2. Proxy → Servidor (socket B): Proxy abre otro socket TCP con el servidor destino, le reenvía el request y recibe la respuesta. Esta vez el proxy actúa de "cliente" para el servidor destino. En el código se ve como "B"
3. Servidor → Proxy: Proxy recibe repuesta del servidor, y modifica la respuesta en caso de ser necesario (ej. Censura, bloqueo) antes de mandarla.
4. Proxy → Cliente: Cliente recibe respuesta del Proxy.

Finalmente el cliente nunca habla directamente con el servidor; todo pasa a través del proxy, que usa **2 sockets**: socket A para comunicarse con el cliente y socket B para comunicarse con el servidor.

Mi actividad 1 está dividida en tres archivos, siendo el principal el proxy.py que define el flujo del programa como fue descrito recién.

1. FUNCIONES AUXILIARES: **UTILS.PY**

El archivo utils.py contiene las funciones de soporte que permiten procesar, interpretar y modificar los mensajes HTTP dentro del proxy. A continuación se describen:

- **parse_HTTP_message**: transforma un mensaje HTTP en un diccionario con campos 'method', 'path', 'version', 'headers' y opcionalmente 'body'. Se usó un diccionario porque me facilita acceder directamente a las llaves y modificar valores como los headers o el body.
- **create_HTTP_message**: recibe el diccionario generado por parse_HTTP_message y lo reconstruye en un mensaje HTTP.
- **receive_full_message**: recibe un mensaje completo desde un socket, leyendo en bucle hasta detectar la secuencia de fin de mensaje. Esto permite manejar buffers más pequeños que el mensaje.
- **contains_end_of_message**: verifica si un mensaje contiene la secuencia de fin.
- **remove_end_of_message**: elimina la secuencia de fin del mensaje completo.
- **is_blocked**: revisa si un path se encuentra dentro de la lista de páginas bloqueadas.
- **count_forbidden_words**: cuenta cuántas veces aparece cada palabra prohibida (definida en el JSON) dentro del HTML.
- **reemplazar_forbidden_words**: reemplaza en el HTML cada palabra prohibida por el valor definido en el JSON.

- **procesar_html**: integra las dos funciones anteriores para detectar palabras prohibidas, reemplazarlas y retornar el HTML modificado.
- **replace_body_http**: reemplaza el body del mensaje HTTP por un nuevo body y actualiza el header Content-Length.
- **recv_http_message**: recibe un mensaje HTTP completo desde un socket, leyendo en bucle hasta obtener los **headers** (detectando \r\n\r\n) y luego acumulando el **body** hasta que el servidor cierre la conexión.

Todas estas están documentadas en el mismo código también.

2. FLUJO DEL PROXY: **PROXY.PY**

El archivo proxy.py define el flujo principal del programa:

1. Inicio:
    a. Carga config.json (usuario, páginas bloqueadas, palabras prohibidas).
    b. Arma socket A en 127.0.0.1:8000 y queda en listen() esperando clientes.
2. **Cliente → Proxy**
    a. Acepta conexión y recibe la request completa con receive_full_message
    b. Parsea la request con parse_HTTP_message a un diccionario.
    c. Revisión de página bloqueada: Extrae path y lo pasa por is_blocked
        i. Si está bloqueado → construye una respuesta 403 (response_blocked_error) y la manda al cliente.
            1. Fin
        ii. Si no → no cambia nada y sigue
3. **Proxy → Servidor (si no está bloqueado)**
    a. Crea socket B, extrae el ip_host de la request del cliente y lo conecta al puerto 80.
    b. Inserta un header (X-ElQuePregunta: PROOOXY) y reenvía la request create_HTTP_message
4. **Servidor → Proxy**
    a. Recibe la respuesta completa con recv_http_message (buffer chico).
    b. Parsea la respuesta y separa el body HTML.
5. Censura (si corresponde)
    a. Cuenta las palabras prohibidas con count_forbidden_words(forbidden_words, body_html).
        i. Si hay matches → procesar_html reemplaza según el diccionario en archivo.json.
        ii. Si no → entonces no hay censura, no cambia nada y sigue
    b. Reconstruye la respuesta con replace_body_http(...), que parsea el mensaje con parse_HTTP_message, actualiza header Content-Length, y la vuelve a transformar con create_HTTP_message(...).
6. **Proxy → Cliente**
    a. Envía la respuesta (censurada o intacta), cierra socket B y luego la conexión con el cliente.
    b. Fin

3. PREGUNTAS FINALES: **BUFFER MUY PEQUEÑO**

1. **¿Cómo sé si llegó el mensaje completo?**
   Sé que los headers llegaron cuando encuentro la secuencia \r\n\r\n, y sé que el mensaje entero (headers+body) llegó cuando recv devuelve vacío (b"") → cuando servidor cierra la conexión-
2. **¿Qué pasa si los headers no caben en mi buffer?**
   Si los headers no caben en el buffer el while sigue llamando a recv y concatenando hasta que aparezca \r\n\r\n. Así los headers se reciben completos aunque lleguen en varios fragmentos.
3. **¿Cómo sé que el HEAD llegó completo?**
   El HEAD está completo cuando aparece la secuencia \r\n\r\n, que marca el final de los headers HTTP.
4. **¿Y el BODY?**
   Se lee esa cantidad exacta de bytes en content-length. En transferencia chunked, se leen todos los chunks hasta el final.